

Chapter 4

Transform Coding

4.1. INTRODUCTION TO TRANSFORM CODING

In Chapter 3 we saw that predictive encoding has some success in reducing the amount of information that needs to be transmitted to accurately represent a video picture. The total savings possible are, however, limited. A major drawback of predictive coding is that the number of elements to be transmitted (pixel differences) is the same after prediction as it was before prediction (pixels). The saving results from the reduction in the entropy of the prediction difference compared to the entropy of the original pixel values. For further gains, we need to reduce the number of elements that are transmitted. This can be achieved by transforming the data into a different domain.

In transform coding, the original pixel values are multiplied by a set of basis functions to produce a set of products. These products are added together to produce the coefficient for that basis function. The coefficient indicates how similar the original pixels are to the particular basis function. If the pixels and the basis function are similar then each product is positive and the result is a large positive value. If the pixels and the basis function are dissimilar then some products are positive whereas others are negative. When summed, the result is close to zero. Finally, if the pixels and the basis functions are similar in shape but different in sign then each of the products is negative and the sum is a large negative value. This is illustrated in Example 4.1.

EXAMPLE 4.1

Consider the simple one-dimensional transform¹ whose basis functions are shown in Figure 4.1 that takes four pixels as input and produces four transform coefficients.

Suppose that the pixel vector to be transformed is [21 8 12 21]. This is shown in Figure 4.2.

¹This is an example of the Hadamard transform.

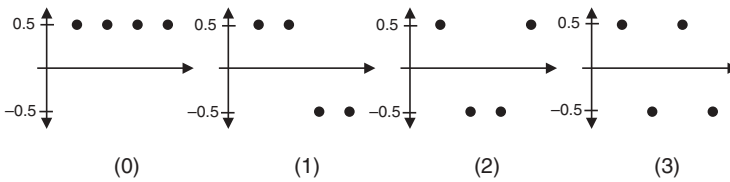


Figure 4.1 Basis functions for a simple four point one-dimensional transform.

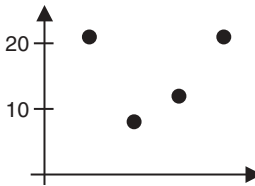


Figure 4.2 Pixel vector to be transformed.

The transform coefficients are then calculated by multiplying the pixel values by the basis functions on a point by point basis and then summing. Thus

$$\text{Coefficient 0} = 21 \times (+0.5) + 8 \times (+0.5) + 12 \times (+0.5) + 21 \times (+0.5) = 31$$

$$\text{Coefficient 1} = 21 \times (+0.5) + 8 \times (+0.5) + 12 \times (-0.5) + 21 \times (-0.5) = -2$$

$$\text{Coefficient 2} = 21 \times (+0.5) + 8 \times (-0.5) + 12 \times (-0.5) + 21 \times (+0.5) = 11$$

$$\text{Coefficient 3} = 21 \times (+0.5) + 8 \times (-0.5) + 12 \times (+0.5) + 21 \times (-0.5) = 2$$

The resulting coefficients are shown in Figure 4.3.

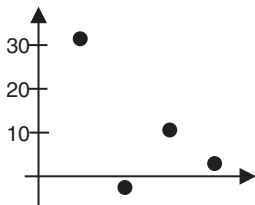


Figure 4.3 Resulting transform coefficients.

The first coefficient gives the average value of the pixels. For this reason it is often called the DC² coefficient. The remaining coefficients give information about the variation of the pixel values and are sometimes referred to as AC³ coefficients. In this case, the second AC coefficient (coefficient 3) is larger than the other two because the general shape of the pixels to be transformed is a close match to the basis function of this coefficient.

The original pixel values can be recovered by the inverse transform process. This consists of weighting each basis function by its coefficient and then summing the four resulting vectors.

²Direct current.

³Alternating current.

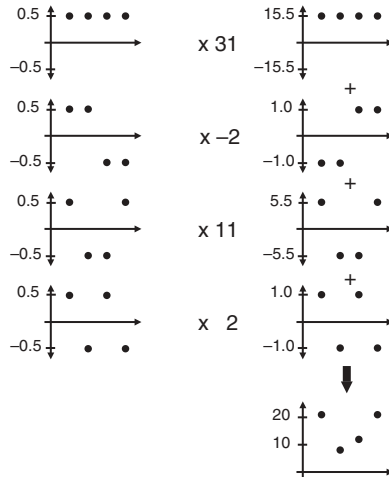


Figure 4.4 Inverse transform process.

$$[+0.5 \quad +0.5 \quad +0.5 \quad +0.5] \times 31 = [+15.5 \quad +15.5 \quad +15.5 \quad +15.5]$$

$$[+0.5 \quad +0.5 \quad -0.5 \quad -0.5] \times -2 = [-1.0 \quad -1.0 \quad +1.0 \quad +1.0]$$

$$[+0.5 \quad -0.5 \quad -0.5 \quad +0.5] \times 11 = [+5.5 \quad -5.5 \quad -5.5 \quad +5.5]$$

$$[+0.5 \quad -0.5 \quad +0.5 \quad -0.5] \times 2 = [+1.0 \quad -1.0 \quad +1.0 \quad -1.0]$$

$$\text{On summing we obtain} \quad [21 \quad 8 \quad 12 \quad 21]$$

This was the original pixel vector. The inverse transformation process is illustrated in Figure 4.4. ■

4.2. THE FOURIER TRANSFORM

A common transform, one that many engineers and scientists are familiar with, is the Fourier transform. Some explanation of how the Fourier transform works and how it represents signals will help in understanding how transforms are used for compression. The Fourier transform is used to take a signal in the time domain and transform it into the frequency domain. This is possible because any time-domain signal can be represented by a weighted sum of sine and cosine waveforms. Thus the periodic time domain signal $f(t)$ with period T_0 can be represented as

$$f(t) = a_0 + \sum_{k=1}^{\infty} a_k \cos n\omega_0 t + \sum_{k=1}^{\infty} b_k \sin n\omega_0 t \quad \text{where } \omega_0 = 2\pi / T_0.$$

The sinusoidal and cosinusoidal signals are called basis functions. In transform coding, the original signal is reproduced when these basis functions (appropriately weighted) are summed.

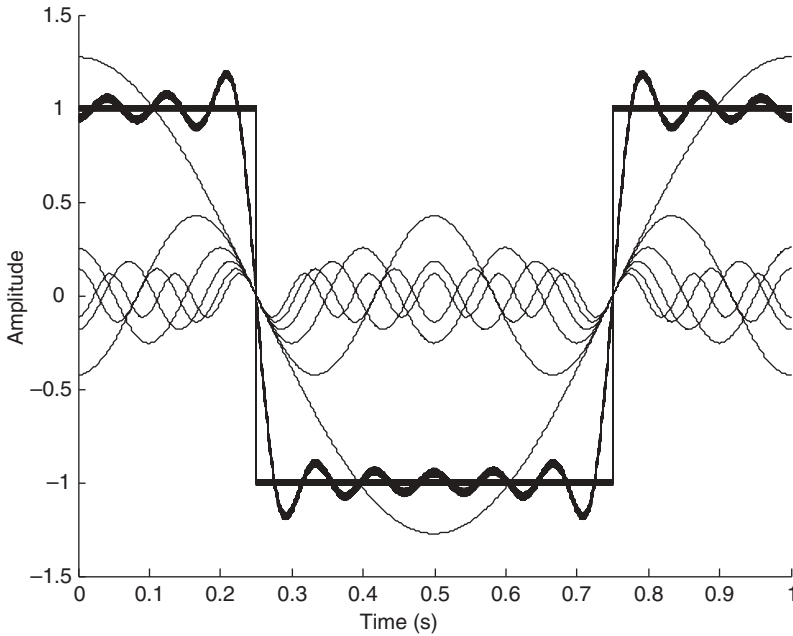


Figure 4.5 Square wave approximated by a weighted summation of cosine waveforms.

Figure 4.5 shows a square wave whose period is 1 s. Fourier analysis shows that the following Fourier series can represent this waveform.

$$f(t) = \frac{4}{\pi} \left(\cos \omega_0 t + \frac{1}{3} \cos 3\omega_0 t + \frac{1}{5} \cos 5\omega_0 t + \frac{1}{7} \cos 7\omega_0 t + \frac{1}{9} \cos 9\omega_0 t + \frac{1}{11} \cos 11\omega_0 t + \dots \right)$$

where $\omega_0 = 2\pi$.

Figure 4.5 also shows the first six terms of the Fourier expansion plus the sum of these six terms. The convergence toward a square wave is apparent.

The weighting factor for each basis function is determined by comparing the basis function to the signal to be transformed. If the signal looks very similar to the basis function then a large weighting factor will result. If the signal and the basis function are dissimilar, a small weighting factor will result.

In the case of the Fourier transform of a periodic signal with period T_0 , the weighting factors are determined according to

$$a_0 = \frac{1}{T_0} \int_0^{T_0} f(t) dt$$

$$a_k = \frac{2}{T_0} \int_0^{T_0} f(t) \cos k\omega_0 t dt, \quad k = 1, 2, \dots, n$$

$$b_k = \frac{2}{T_0} \int_0^{T_0} f(t) \sin k\omega_0 t \, dt, \quad k = 1, 2, \dots, n$$

where $\omega_0 = 2\pi/T_0$

The a_0 term gives the average value of the periodic signal whereas the a_k and b_k values measure the similarity of the waveform to appropriate cosinusoidal and sinusoidal signals, respectively.

The Fourier transform is designed to represent continuous functions. The pictures that we wish to encode are, of course, sampled. In this case, the appropriate transform is the discrete Fourier transform. The method of calculating weighting coefficients of the sampled function $f(n)$ with a total of N sample points is given below.

$$a_k = \sum_{n=0}^{N-1} f(n) \cos\left(\frac{2\pi kn}{N}\right)$$

$$b_k = \sum_{n=0}^{N-1} f(n) \sin\left(\frac{2\pi kn}{N}\right)$$

The sampled values of the basis functions are called basis vectors. Figure 4.6 shows the first cosinusoidal basis vector when $N = 8$. This is a sampled cosine waveform with period eight samples.

The Fourier transform, though an important tool in understanding communication systems, is not the best choice for transform encoding. One reason for this is

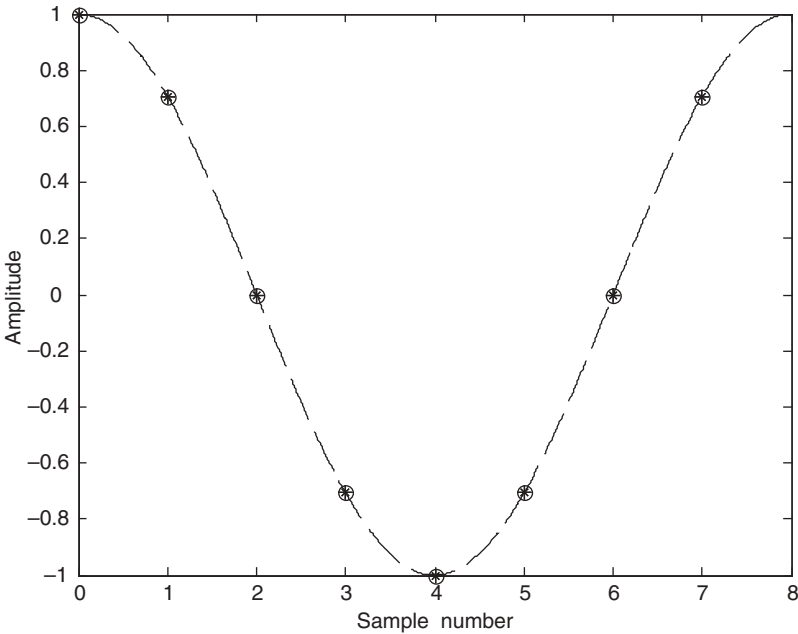


Figure 4.6 First cosine basis vector of discrete Fourier transform.

that there are two weighting factors (a_k and b_k) required for each frequency.⁴ This requires a substantial amount of computation.

4.3. THE KARHUNEN-LOEVE TRANSFORM

The aim of a good signal compression transform is to pack the information in the data into the smallest number of transform coefficients. The data to be transformed is collected into a block. For an N point transform, the data would consist of an N element vector.

$$\mathbf{x} = [x_1 \ x_2 \ \dots \ x_N]$$

The transform consists of multiplying the data vector by an $N \times N$ transform matrix.

$$\mathbf{C} = \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_N \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} & \dots & t_{1N} \\ t_{21} & t_{22} & \dots & t_{2N} \\ \vdots & \vdots & \dots & \vdots \\ t_{N1} & t_{N2} & \dots & t_{NN} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} = \mathbf{T}\mathbf{x}$$

Each row of matrix $\mathbf{T}$ ($\mathbf{t}_m$) represents a basis vector of the transform. An effective transform would produce transform coefficients $\mathbf{C}$ that pack the information contained in the data into as few transform coefficients as possible. This means that it is not necessary to transmit all of the coefficients to obtain a satisfactory reconstruction of the original picture or, if the coefficients are very small, they may not need to be represented very accurately. The lower the number of coefficients that need to be transmitted, the less the amount of data required to transmit them.

Transforms are always orthonormal,⁵ that is, when two basis vectors are multiplied together and summed

$$\mathbf{t}_m^T \mathbf{t}_n = \begin{cases} 1, & m = n \\ 0, & m \neq n \end{cases}$$

where the superscript T represents the matrix transpose.

It follows directly that the inverse transform ($\mathbf{T}^{-1}$) is simply the transpose of the forward transform ($\mathbf{T}^T$).

$$\mathbf{T}^{-1} = \mathbf{T}^T$$

⁴Many texts on digital signal processing refer to the discrete Fourier transform as generating a complex value for each weighting coefficient of the form $a_k + jb_k$.

⁵This ensures that if we initially approximate a desired function with a limited number of weighted basis functions, the subsequent addition of further weighted basis functions to improve the approximation will not change the values of the coefficients weighting the original basis functions.

We can therefore write

$$\mathbf{x} = \mathbf{T}^{-1}\mathbf{C} = \mathbf{T}^T\mathbf{C} = \begin{bmatrix} t_{11} & t_{21} & \dots & t_{N1} \\ t_{12} & t_{22} & \dots & t_{N2} \\ \vdots & \vdots & \dots & \vdots \\ t_{1N} & t_{2N} & \dots & t_{NN} \end{bmatrix} \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_N \end{bmatrix}$$

and so

$$x = \sum_{k=1}^N C_k t_k = C_1 t_1 + C_2 t_2 + \dots + C_N t_N$$

Thus the original signal can be thought of as a weighted sum of the basis vectors. These weights, $\mathbf{C}$, are calculated by the forward transform.

If we want to understand the theory behind transform coding of pictures, some statistical measures of pictures need to be defined. The mean of an $N \times M$ pixel picture is given by

$$\bar{x} = \frac{1}{NM} \sum_{n=1}^N \sum_{m=1}^M x_{n,m}$$

where $x_{n,m}$ is the value of pixel m on line n of the picture. The expected value operator $E(\cdot)$ is often used to define an averaging operation. We can therefore write

$$\bar{x} = E(x)$$

The variance of an $N \times M$ pixel picture is defined as

$$\sigma^2 = \frac{1}{NM} \sum_{n=1}^N \sum_{m=1}^M (x_{n,m} - \bar{x})^2$$

which can be written using the expected value operator as

$$\sigma^2 = E((x - \bar{x})^2)$$

If we have two random variables, x_1 and x_2 , the covariance of the two variables can be written as

$$\sigma_{12}^2 = E((x_1 - \bar{x})(x_2 - \bar{x}))$$

Using this terminology, the variance of a single variable would be referred to as σ_{11} . If we have k random variables, the covariance matrix $\mathbf{R}_x$ is defined as

$$\mathbf{R}_x = \begin{bmatrix} \sigma_{11}^2 & \sigma_{12}^2 & \dots & \sigma_{1k}^2 \\ \sigma_{21}^2 & \sigma_{22}^2 & \dots & \sigma_{2k}^2 \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{k1}^2 & \sigma_{k2}^2 & \dots & \sigma_{kk}^2 \end{bmatrix}$$

Consider a single line of pixels in a picture as defined by

$$x_{1,1} \quad x_{1,2} \quad x_{1,3} \quad \dots \quad x_{1,m}$$

Let random variable x_1 and x_2 represent the pixel values

$$x_1 = x_{1,1} \quad x_{1,2} \quad x_{1,3} \quad \dots \quad x_{1,m-1}$$

$$x_2 = x_{1,2} \quad x_{1,3} \quad x_{1,4} \quad \dots \quad x_{1,m}$$

The covariance for these two random variables is given by

$$\sigma_{12}^2 = E((x_1 - \bar{x}_1)(x_2 - \bar{x}_2))$$

and is called the one-step horizontal covariance of the picture line. The k -step horizontal covariance is defined for the random variables x_1 and x_{k+1} that represent the pixel values

$$\begin{array}{ccccccc} x_1 = x_{1,1} & x_{1,2} & x_{1,3} & \dots & x_{1,m-k} \\ x_{k+1} = x_{1,k+1} & & x_{1,k+2} & x_{1,k+3} & \dots & x_{1,m} \end{array}$$

and is given by

$$\sigma_{1k}^2 = E((x_1 - \bar{x}_1)(x_{k+1} - \bar{x}_{k+1}))$$

If we want to calculate the covariance matrix for a whole picture, the covariance of each line is calculated using the mean value of the entire picture rather than the mean value for each variable on each line. The k -step horizontal covariance for each line is then given by

$$\sigma_{1k}^2 = E((x_1 - \bar{x})(x_{k+1} - \bar{x}))$$

where $\bar{x}$ is the mean of the entire image.

The k -step horizontal covariance for the picture is the average of the covariance for each line of the picture. A vertical covariance matrix can also be produced using pixel values down columns as opposed to across rows in the picture. For the first luminance picture in the “Mobile and Calendar” sequence, the 4×4 horizontal covariance matrix (rounded to the nearest integer) is given by

$$\begin{bmatrix} 3914 & 3525 & 3126 & 3050 \\ 3525 & 3907 & 3518 & 3118 \\ 3126 & 3518 & 3901 & 3511 \\ 3050 & 3118 & 3511 & 3894 \end{bmatrix}$$

By similar reasoning, the 4×4 vertical covariance matrix (rounded to the nearest integer) for this picture is given by

$$\begin{bmatrix} 3914 & 3540 & 3312 & 3031 \\ 3540 & 3914 & 3541 & 3313 \\ 3312 & 3541 & 3915 & 3541 \\ 3031 & 3313 & 3541 & 3915 \end{bmatrix}$$

As stated earlier, the aim of a good transform is to pack the information content of the data into the smallest possible number of transform coefficients. Although information content is somewhat difficult to quantify, one approach to achieving this aim would be to pack the maximum amount of energy into the smallest number of transform coefficients. If we represent the original data by only the first M of N transform coefficients with the remaining coefficients replaced by a constant a_j then we obtain

$$\hat{\mathbf{x}}_{\mathbf{M}} = \sum_{k=1}^M C_k \mathbf{t}_k + \sum_{k=M+1}^N a_k \mathbf{t}_k$$

The resulting error is

$$\begin{aligned} \mathbf{x}_{\text{eM}} &= \mathbf{x} - \hat{\mathbf{x}}_{\mathbf{M}} \\ &= \sum_{k=M+1}^N C_k \mathbf{t}_k - \sum_{k=M+1}^N a_k \mathbf{t}_k \\ &= \sum_{k=M+1}^N (C_k - a_k) \mathbf{t}_k \end{aligned}$$

We now want to calculate $e_{\mathbf{M}}$ the average energy in $\mathbf{x}_{\text{eM}}$ where

$$\begin{aligned} e_{\mathbf{M}} &= E \left\{ \left[\sum_{k=M+1}^N (C_k - a_k) \mathbf{t}_k \right]^2 \right\} \\ &= E \left\{ [(C_{M+1} - a_{M+1}) \mathbf{t}_{M+1} + (C_{M+2} - a_{M+2}) \mathbf{t}_{M+2} + \dots + (C_N - a_N) \mathbf{t}_N]^2 \right\} \\ &= E \left\{ (C_{M+1} - a_{M+1})^2 \mathbf{t}_{M+1}^T \mathbf{t}_{M+1} + (C_{M+1} - a_{M+1})(C_{M+2} - a_{M+2}) \mathbf{t}_{M+1}^T \mathbf{t}_{M+2} + \dots \right. \\ &\quad \left. + (C_N - a_N)(C_N - a_N) \mathbf{t}_N^T \mathbf{t}_N \right\} \end{aligned}$$

As the transform is orthonormal

$$\mathbf{t}_i^T \mathbf{t}_j = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

Thus the equation for $e_{\mathbf{M}}$ simplifies to

$$e_{\mathbf{M}} = E \left(\sum_{k=M+1}^N (C_k - a_k)^2 \right)$$

We can now choose the value of a_k that minimizes $e_{\mathbf{M}}$ by partial differentiation with respect to each a_j where j is in the range $M+1$ to N .

$$\begin{aligned} \frac{\partial}{\partial a_j} E \left(\sum_{k=M+1}^N (C_k - a_k)^2 \right) &= E(-2(C_j - a_j)) \\ &= -2(E(C_j) - E(a_j)) \\ &= -2(E(C_j) - a_j) \\ &= 0, \text{ at the minimum} \end{aligned}$$

Therefore, at the minimum

$$a_j = E(C_j)$$

Now since

$$C_j = \mathbf{t}_j^T \mathbf{x}$$

It follows that

$$\begin{aligned} a_j &= E(\mathbf{t}_j^T \mathbf{x}) \\ &= \mathbf{t}_j^T E(\mathbf{x}) \end{aligned}$$

$E(\mathbf{x})$ is the mean vector of the data. Setting the mean value to zero results in the optimum value a_j ($j = M + 1$ to N) being zero and so

$$\hat{\mathbf{x}}_M = \sum_{k=1}^M C_k \mathbf{t}_k$$

From the above, it follows that

$$C_k - a_k = \mathbf{t}_k^T (\mathbf{x} - E(\mathbf{x}))$$

This can be substituted into

$$e_M = E \left(\sum_{k=M+1}^N (C_k - a_k)^2 \right)$$

to obtain

$$\begin{aligned} e_M &= E \left(\sum_{k=M+1}^N (\mathbf{t}_k^T (\mathbf{x} - E(\mathbf{x})))^2 \right) \\ &= E \left(\sum_{k=M+1}^N \mathbf{t}_k^T (\mathbf{x} - E(\mathbf{x})) (\mathbf{x} - E(\mathbf{x}))^T \mathbf{t}_k \right) \\ &= \sum_{k=M+1}^N \mathbf{t}_k^T E((\mathbf{x} - E(\mathbf{x}))(\mathbf{x} - E(\mathbf{x}))^T) \mathbf{t}_k \\ &= \sum_{k=M+1}^N \mathbf{t}_k^T (\text{COV}(\mathbf{x})) \mathbf{t}_k \end{aligned}$$

where $\text{COV}(\mathbf{x})$ is the covariance matrix of the input data.

We now need to minimize e_M with respect to the basis vectors $\mathbf{t}_k$ while maintaining the orthonormal property. Using Lagrange multipliers, we therefore minimize

$$\begin{aligned} e'_M &= e_M - \sum_{k=M+1}^N \lambda_k (\mathbf{t}_k^T \mathbf{t}_k - 1) \\ &= \sum_{k=M+1}^N \mathbf{t}_k^T \text{COV}(\mathbf{x}) \mathbf{t}_k - \lambda_k (\mathbf{t}_k^T \mathbf{t}_k - 1) \end{aligned}$$

Because we want to minimize e'_M , we want to find the point where the gradient of e'_M with respect to $\mathbf{t}_j$ is zero. Noting⁶ that

$$\text{GRAD}_{\mathbf{t}_j}(\mathbf{t}_k^T[A]\mathbf{t}_k) = 2[A]\mathbf{t}_k$$

and

$$\text{GRAD}_{\mathbf{t}_j}(\mathbf{t}_k^T\mathbf{t}_k) = 2\mathbf{t}_k$$

It follows that

$$\text{GRAD}_{\mathbf{t}_j}e'_M = 2\text{COV}(x)\mathbf{t}_j - \lambda_k(2\mathbf{t}_j) = 0$$

from which it follows that

$$\text{COV}(x)\mathbf{t}_k = \lambda_k\mathbf{t}_j$$

which when solved for $\mathbf{t}_k$ and λ_k will lead to the k th basis vector of the optimum transform. This is exactly the eigenvalue equation, and so $\mathbf{t}_k$ is an eigenvector of the covariance matrix whereas λ_k is the corresponding eigenvalue. Because the covariance matrix is positive and symmetrical about the leading diagonal, it will always have eigenvectors and eigenvalues that will be real.

If the eigenvalues are arranged from largest to smallest this optimum transform, which is called the Karhunen–Loeve transform (KLT),⁷ packs the maximum amount of energy into any given number of coefficients. Putting this another way, the mean square error introduced by deleting a given number of coefficients is a minimum for this transform.

EXAMPLE 4.2—MATLAB

Calculate the 8×8 horizontal covariance matrix for the first luminance picture in the “Mobile and Calendar” sequence. Calculate the eigenvectors for this matrix. Hence determine the percentage of energy in each coefficient.

SOLUTION The 8×8 horizontal covariance matrix can be calculated using the following MATLAB function.

```
function covar = covariance_h(PICTURE)
[row,col] = size(PICTURE);
PICTURE = PICTURE - mean(mean(PICTURE));
Covar = Zeros(8,8);
Sum = 0;

For matrix_row = 1:8
    For matrix_col = 1:8
        delta = matrix_col-matrix_row;
```

⁶See R.J. Clarke, *Transform Coding of Pictures*, Academic Press, 1985 Appendix 4 for further details.

⁷It is also sometimes referred to as the Hotelling transform or the principal components transform.

```

    If delta ≤ 0
        col_max = col;
    else
        col_max = col - delta;
    end
    for row_num = 1:row
        for col_num = matrix_row:col_max
            sum = sum + PICTURE(row_num,col_num)*PICTURE(row_num,col_num+delta);
        end
        sum = sum/(col_max-matrix_row+1);
        covar(matrix_row,matrix_col) = covar(matrix_row,matrix_col) + sum;
        sum = 0;
    end
end
end
covar = covar/row;

```

When applied to the first picture of the “Mobile and Calendar” sequence, the covariance matrix obtained (after rounding to the nearest integer) is

$$\begin{bmatrix}
 3914 & 3525 & 3126 & 3050 & 2956 & 2883 & 2797 & 2730 \\
 3525 & 3907 & 3518 & 3118 & 3042 & 2948 & 2875 & 2789 \\
 3126 & 3518 & 3901 & 3511 & 3110 & 3035 & 2940 & 2867 \\
 3050 & 3118 & 3511 & 3894 & 3504 & 3102 & 3027 & 2933 \\
 2956 & 3042 & 3110 & 3504 & 3888 & 3496 & 3095 & 3019 \\
 2883 & 2948 & 3035 & 3102 & 3496 & 3881 & 3489 & 3087 \\
 2797 & 2875 & 2940 & 3027 & 3095 & 3489 & 3874 & 3482 \\
 2730 & 2789 & 2867 & 2933 & 3019 & 3087 & 3482 & 3868
 \end{bmatrix}$$

The MATLAB command

$$[V,D] = \text{eig}(A)$$

produces the eigenvalues in matrix D and the corresponding eigenvectors as the columns of matrix V. MATLAB arranges the eigenvectors vertically down columns. It also arranges the eigenvalues so that each is larger than those that come before it whereas it is usual to list the eigenvalues in decreasing order of magnitude. For consistency with the definition of basis vectors given earlier, it is necessary to employ the MATLAB command

$$V = \text{fliplr}(V)';$$

which flips the columns of the matrix from left to right and then calculates the transpose.

When all this is done with the horizontal covariance matrix shown above, the eigenvector matrix, as determined in MATLAB, is

$$\begin{bmatrix} +0.344 & +0.354 & +0.359 & +0.361 & +0.360 & +0.357 & +0.352 & +0.341 \\ -0.426 & -0.448 & -0.317 & -0.115 & +0.115 & +0.318 & +0.452 & +0.430 \\ -0.394 & -0.274 & +0.144 & +0.495 & +0.499 & +0.151 & -0.273 & -0.397 \\ -0.406 & -0.039 & +0.484 & +0.318 & -0.308 & -0.487 & +0.033 & +0.407 \\ -0.397 & +0.253 & +0.434 & -0.299 & -0.306 & +0.429 & +0.258 & -0.396 \\ +0.357 & -0.441 & -0.010 & +0.423 & -0.419 & +0.002 & +0.442 & -0.356 \\ -0.264 & +0.486 & -0.404 & +0.175 & +0.180 & -0.406 & +0.484 & -0.263 \\ -0.157 & +0.318 & -0.405 & +0.460 & -0.459 & +0.404 & -0.316 & +0.156 \end{bmatrix}$$

The eight eigenvectors are plotted in Figure 4.7. We see that the frequency of the eigenvectors increases from top to bottom in the figure.

The energy compaction capability of the transform is simply calculated from the eigenvalues. For the eigenvectors shown in Figure 4.7, the corresponding eigenvalues together with their relative energy are listed in Table 4.1. Note that most of the energy is compacted into the first eigenvalue with about 96% of the total energy compacted into the first four eigenvalues.

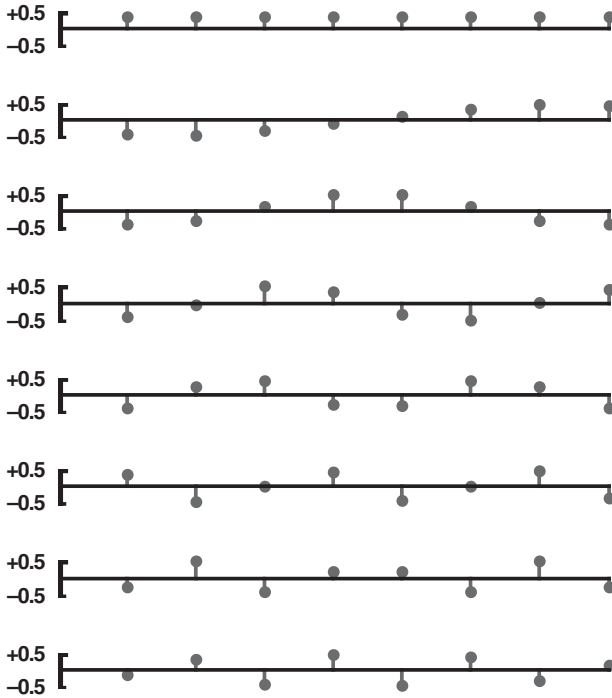


Figure 4.7 Horizontal eigenvectors for the first picture in the “Mobile and Calendar” sequence.

Table 4.1 Horizontal eigenvalues for the first picture in the “Mobile and Calendar” sequence.

Eigenvalue number	Eigenvalue	Percentage of energy
1	25,664	82.4
2	2,104	6.8
3	1,224	3.9
4	877	2.8
5	615	2.0
6	383	1.2
7	203	0.7
8	56	0.2

■

4.4. THE DISCRETE COSINE TRANSFORM

Although the Karhunen–Loeve transform is the optimum transform in terms of energy compaction, it suffers from the significant difficulty that the transform needs to be defined for each picture (or even each 8×8 block within a picture). This requires a significant amount of computation both to calculate the covariance matrix and then the eigenvectors that are used as the basis vectors of the transform. In addition, the transform basis functions (eigenvectors) required for each picture (or each 8×8 block within a picture) need to be transmitted to the decoder so that the picture can be correctly decoded. This represents a significant overhead.

For this reason, a fixed transform known as the discrete cosine transform (DCT) is commonly used in picture and video coding applications. Although suboptimal when compared to the Karhunen–Loeve transform, it has the advantage that it can be used without the need for calculating covariance matrices and eigenvectors. In addition, there is no need to transmit information on the basis vectors used to the receiver.

The basis vectors of an N -point DCT in one-dimension are defined as

$$C(u) = \alpha(u) \sum_{x=0}^{N-1} f(x) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \quad u = 0, 1, 2, \dots, N-1$$

Similarly, the inverse DCT is defined as

$$f(x) = \sum_{u=0}^{N-1} \alpha(u) C(u) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \quad x = 0, 1, 2, \dots, N-1$$

In both of these equations

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for } u = 0 \\ \sqrt{\frac{2}{N}} & \text{for } u = 1, 2, \dots, N-1 \end{cases}$$

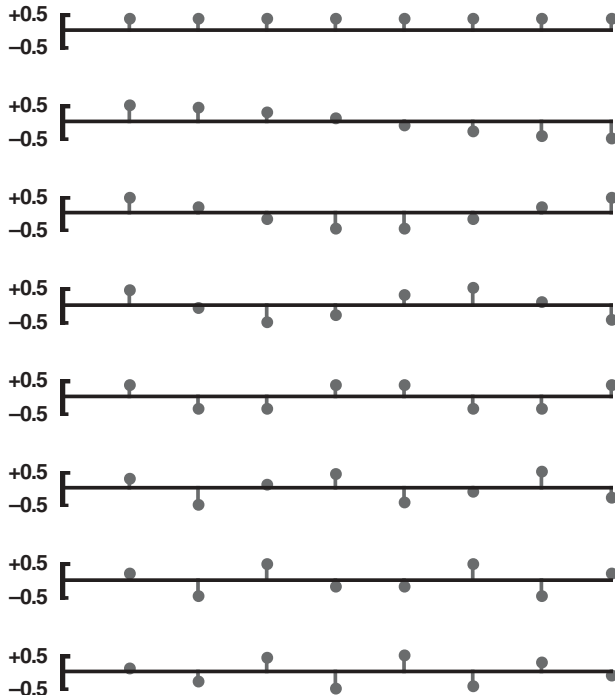
$$\begin{bmatrix} +0.354 & +0.354 & +0.354 & +0.354 & +0.354 & +0.354 & +0.354 & +0.354 \\ +0.490 & +0.416 & +0.278 & +0.098 & -0.098 & -0.278 & -0.416 & -0.490 \\ +0.462 & +0.191 & -0.191 & -0.462 & -0.462 & -0.191 & +0.191 & +0.462 \\ +0.416 & -0.098 & -0.490 & -0.278 & +0.278 & +0.490 & +0.098 & -0.416 \\ +0.354 & -0.354 & -0.354 & +0.354 & +0.354 & -0.354 & -0.354 & +0.354 \\ +0.278 & -0.490 & +0.098 & +0.416 & -0.416 & -0.098 & +0.490 & -0.278 \\ +0.191 & -0.462 & +0.462 & -0.191 & -0.191 & +0.462 & -0.462 & +0.191 \\ +0.098 & -0.278 & +0.416 & -0.490 & +0.490 & -0.416 & +0.278 & -0.098 \end{bmatrix}$$

Figure 4.8 Basis functions for the DCT.

The basis vectors for an eight-point DCT are given in the matrix shown in Figure 4.8. They are also shown in Figure 4.9.

These are just sampled versions of cosine waveforms of increasing frequency ranging from 0 periods per vector (i.e., constant) in the case of the first vector to 3.5 periods per vector in the case of the last vector with each vector containing 0.5 additional periods to the one before it.

Comparing the eigenvectors from the Karhunen–Loeve transform shown in Figure 4.7 with those for the DCT shown in Figure 4.9, we note a remarkable

**Figure 4.9** Basis vectors for the eight-point discrete cosine transform.

similarity in general shape. The most significant difference is that several of the corresponding eigenvectors in each figure are approximately the negative of each other. However, the effect of changing the sign of an eigenvector is just that the sign of the associated transform coefficients will be changed—energy compaction is identical. The DCT then seems to be a reasonable approximation to the Karhunen–Loeve transform at least for this picture.

For pictures, a two-dimensional DCT is required. The forward and reverse transform are calculated according to the equations

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \left[\frac{(2y+1)v\pi}{2N} \right] \quad u, v = 0, 1, 2, \dots, N-1$$

and

$$f(x, y) = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} \alpha(u)\alpha(v) C(u, v) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \left[\frac{(2y+1)v\pi}{2N} \right] \quad x, y = 0, 1, 2, \dots, N-1$$

where again

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{N}} & \text{for } u = 0 \\ \sqrt{\frac{2}{N}} & \text{for } u = 1, 2, \dots, N-1 \end{cases}$$

The basis vectors $t(u, v)$ in this case are two-dimensional arrays defined by

$$t(u, v) = \alpha(u)\alpha(v) \cos \left[\frac{(2x+1)u\pi}{2N} \right] \left[\frac{(2y+1)v\pi}{2N} \right] \quad \text{for } x, y = 0, 1, \dots, N-1$$

Pictures of the 64 two-dimensional basis vectors from an 8×8 transform are shown in Figure 4.10 that includes an offset so that negative values appear dark and positive values appear light with mid-gray representing values close to zero. Vertical frequency increases from top to bottom whereas horizontal frequency increases from left to right. Any 8×8 pixel block can be represented by a weighted sum of these two-dimensional basis vectors.

Fortunately, the DCT is a separable transform. This means that the two-dimensional transform can be obtained by first applying a one-dimensional transform across the rows of the data. The result after the horizontal transform has a one-dimensional transform applied vertically to yield the final two-dimensional transform result. This greatly simplifies the transform procedure as well as significantly increasing the speed of the transform.

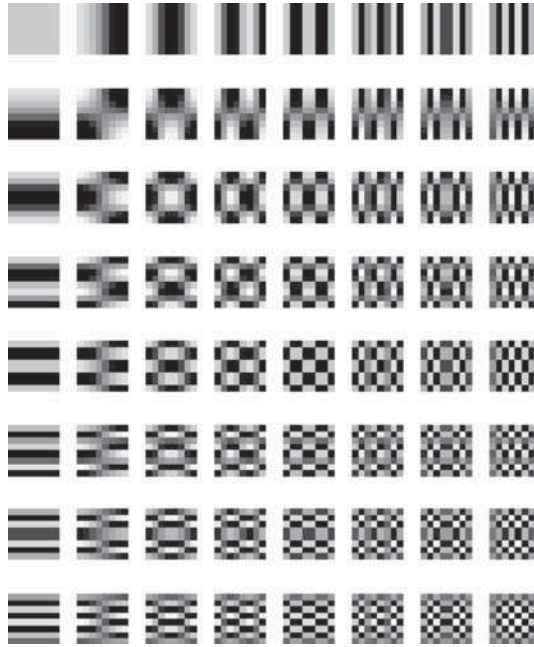


Figure 4.10 Basis vectors for two-dimensional discrete cosine transform.

EXAMPLE 4.3—MATLAB

For the 8×8 pixel block shown in Figure 4.11, calculate the two-dimensional DCT. Hence, show that the picture can be represented by a weighted sum of the two-dimensional basis vectors shown in Figure 4.10.

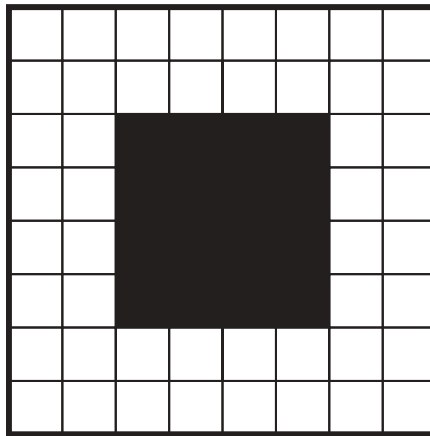


Figure 4.11 Block to be analyzed in Example 4.3.

This picture is represented by the 8×8 pixel array of data shown in Figure 4.12 where white is represented by 255 and black by 0.

255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255
255	255	0	0	0	0	255	255
255	255	0	0	0	0	255	255
255	255	0	0	0	0	255	255
255	255	0	0	0	0	255	255
255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255

Figure 4.12 Numerical picture data for Example 4.3.

As explained earlier, the two-dimensional DCT can be calculated by first taking the one-dimensional DCT horizontally and then vertically. After applying the horizontal DCT and rounding to the nearest integer we obtain the result shown in Figure 4.13.

+721	0	0	0	0	0	0	0
+721	0	0	0	0	0	0	0
361	0	333	0	0	0	-138	0
361	0	333	0	0	0	-138	0
361	0	333	0	0	0	-138	0
361	0	333	0	0	0	-138	0
+721	0	0	0	0	0	0	0
+721	0	0	0	0	0	0	0

Figure 4.13 Data of Figure 4.12 after one-dimensional horizontal DCT.

The final two-dimensional DCT is obtained by repeating the one-dimensional discrete cosine transform down the columns of this result. The final result is shown in Figure 4.14.

+1530	0	471	0	0	0	-195	0
0	0	0	0	0	0	0	0
+471	0	-435	0	0	0	+180	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
-195	0	+180	0	0	0	-75	0
0	0	0	0	0	0	0	0

Figure 4.14 Data of Figure 4.12 after two-dimensional DCT.

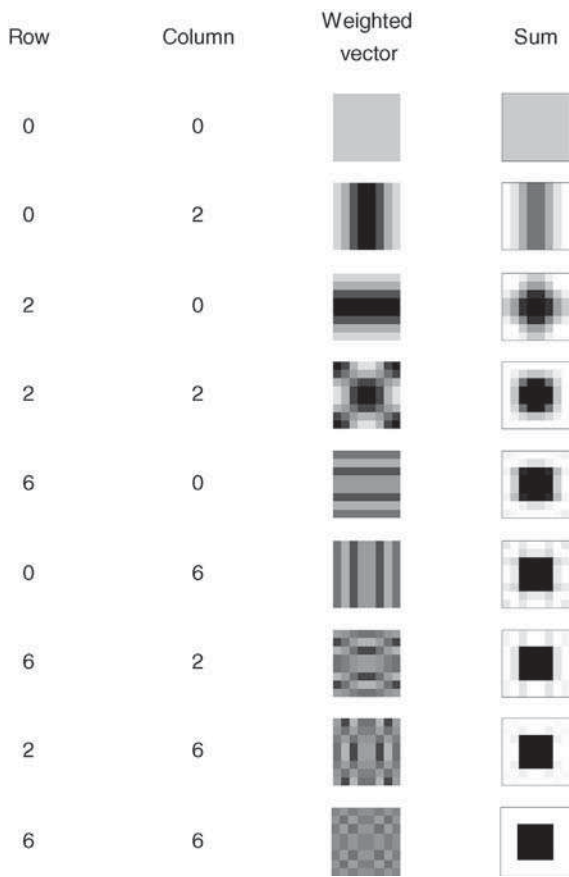


Figure 4.15 Weighted two-dimensional basis vectors for the block of Figure 4.11 together with the result from summing the weighted basis vectors to reconstruct the original block.

The weighted two-dimensional basis vectors for each nonzero term in the two-dimensional DCT are shown in Figure 4.15. Also shown is the result when the current weighted basis vector is added to the sum of all the weighted basis vectors appearing above it in Figure 4.15. When all the weighted basis vectors have been summed, we end up with the original block. The last few basis vectors make only a small change to the final block despite the fact that the block chosen has a number of sharp discontinuities that usually imply significant energy at high frequencies. A smoother block would show even less distortion when high frequency basis vectors were omitted. ■

4.4.1. Choice of Transform Block Size

Example 4.3 uses a block size of 8×8 pixels. The appropriate block size is a compromise between the amount of compression achieved (which tends to increase with block size), the correlation within the picture (which tends to decrease with block size), the ability to adapt to local picture statistics (which is better as block size

decreases), and computational complexity (which increases with block size). The block size is invariably chosen to be a power of two (i.e., 4×4 , 8×8 , and 16×16 pixels) as this simplifies computational complexity.

EXAMPLE 4.4—MATLAB

For the first picture of the “Mobile and Calendar” sequence, divide the picture into square blocks of size $N = 2, 4, 8, 16, 32$, and 64 pixels and calculate the DCT of each block. Now retain only the top $N/2 \times N/2$ pixels and calculate the RMS error for each reconstructed picture. Hence, comment on the most appropriate choice of transform block size.

The MATLAB function `dct2(A)` will calculate the two-dimensional DCT of a block of data. Appropriate MATLAB code to perform this task is given below. The DCT block size is set by the variable `tf_size`.

```
tf_size = 8;
for irow = 1:tf_size:row
    for icol = 1:tf_size:col
        dct_block = dct2(A(irow:irow1(tf_size-1),icol:icol+(tf_size-1)));
        limit = (tf_size/2)+1;
        dct_block(limit:tf_size,:) = zeros((tf_size/2),tf_size);
        dct_block(:,limit:tf_size) = zeros(tf_size,(tf_size/2));
        rec(irow:irow+(tf_size-1),icol:icol+(tf_size-1)) = round(idct2(dct_block));
    end
end
rms = sqrt(mean(mean((A - rec). * (A-rec))));
```

The result when applied to the picture is shown in Figure 4.16.

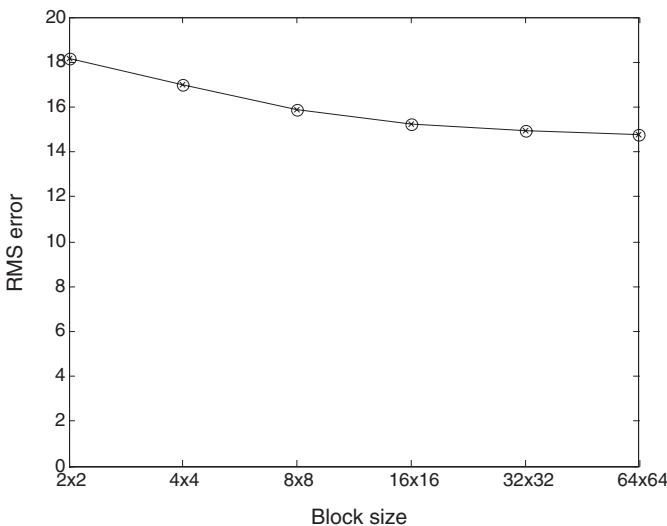


Figure 4.16 Result of deleting all but the top left quarter coefficients for the first picture of the “Mobile and Calendar” sequence.

It is clear that most of the savings are achieved by the time a block size between 8×8 and 16×16 pixels is reached. Hardware complexity considerations lead to the choice of an 8×8 pixel block size. ■

4.4.2. Quantization of DCT Transform Coefficients

We have now succeeded in transforming integer pixel values into real transform coefficients. Transmitting these coefficients without any further processing would probably lead to an increase in the number of bits of information required to represent the picture. However, the transform has packed most of the energy of the picture into a small number of coefficients. Quantizing these coefficients and then transmitting only the significant ones can result in a significant saving. The question remains how best to do this. As the energy is compacted primarily into the first few (low frequency) coefficients, one approach would be to simply not transmit a number of the other (high frequency) coefficients. This is considered in Example 4.5.

EXAMPLE 4.5—MATLAB

For the first luminance picture in the sequence “Mobile and Calendar,” calculate the resulting picture when only the top left 4×4 , 2×2 , and 1×1 DCT coefficients are retained.

The results are shown in Figure 4.17. Even retaining the top 4×4 low-frequency coefficients (Fig. 4.17a) leads to significant blurring in the reconstructed picture. Reducing this



(a)

Figure 4.17 Effect of deleting high-frequency DCT coefficients: (a) top 4×4 coefficients retained; (b) top 2×2 coefficients retained; (c) top 1×1 coefficient retained.



(b)



(c)

Figure 4.17 (Continued)

to the top 2×2 low-frequency coefficients (Fig. 4.17b) greatly increases the blurring. In addition, the edges of the individual 8×8 pixel blocks start to become obvious. When only the single DC coefficient is retained (Fig. 4.17c) then the picture becomes a series of blocks. This is hardly surprising as retaining only the DC coefficient means that each pixel in the 8×8 pixel block is replaced by the average value of the block. ■

The previous example has demonstrated that performing the DCT and then simply deleting the higher frequency coefficients is not a satisfactory approach if high-quality reconstructed pictures are required. Although low-frequency information is almost always important, simply removing high-frequency information leads to blurring at sharp edges where high-frequency information is significant.

After quantization, it is desirable that the maximum number of coefficients are zero as this reduces the amount of information that needs to be transmitted. For this reason, a quantizer with a larger than normal “dead zone” (i.e., a quantization region where the coefficient will be set to zero) as shown in Figure 4.18 is commonly employed.

By comparison, a completely linear quantizer would have decision levels at $\dots -2.5Q, -1.5Q, -0.5Q, +0.5Q, +1.5Q, +2.5Q \dots$ and reconstruction levels at $\dots -2Q, -Q, 0, +Q, +2Q \dots$. The larger dead zone ensures that all coefficients in the range $-Q$ to Q are set to zero. The value of the quantizer (Q) is chosen by the user to ensure an adequate representation of the picture. More is said on this topic later.

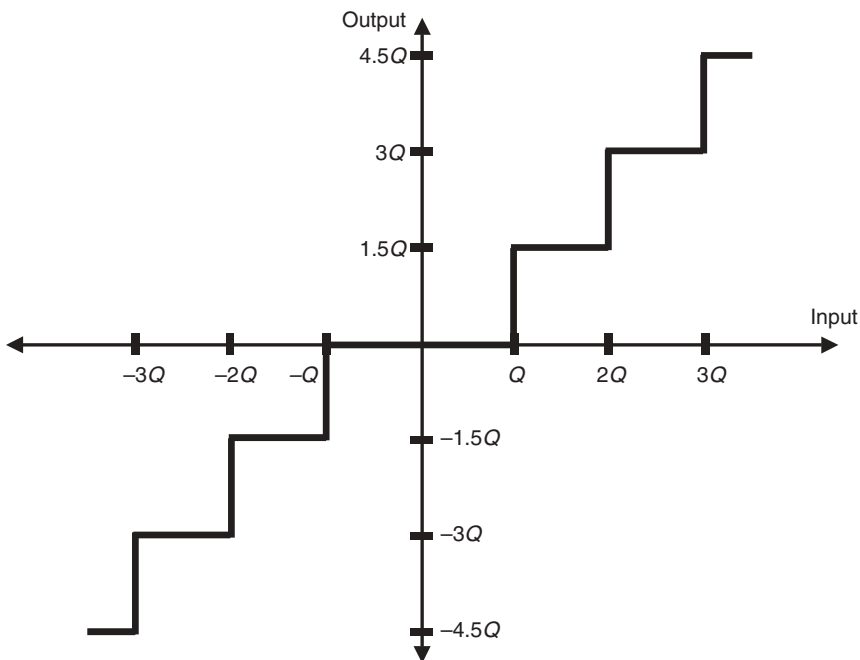


Figure 4.18 Quantizer with central dead zone.

4.4.3. Quantization of DCT Coefficients Based on the Human Visual System

Despite the results shown in Example 4.5, it is well known that the sensitivity of the human visual system does indeed decrease as the spatial frequency (usually measured in cycles per degree of arc) increases. Figure 4.19 shows an indicative plot of the relative spatial frequency response of the eye as a function of spatial frequency measured in cycles per degree of sight.

It is clear that the response peaks at a spatial frequency around 5–10 cycles/degree and falls off sharply at higher frequencies. However, even at these higher frequencies a significant signal will still be observable.

Almost invariably, the transform coefficients are quantized by a linear or near linear quantizer. However, the step size of the quantizer can be varied according to the spatial frequency represented with the step size increasing as the spatial frequency increases. This ensures that high-frequency coefficients will be quantized to zero unless they are sufficiently large that they are likely to be observable to the human visual system.

One way that this can be done is to use a quantization relationship such as

$$\hat{C}_{i,j} = \text{round} \left(\frac{8 \times C_{i,j}}{Q \times W_{i,j}} \right)$$

where $\hat{C}_{i,j}$ is the value of the quantized transform coefficient, $C_{i,j}$ is the value of the original transform coefficient, Q is the quantizer step size for a particular

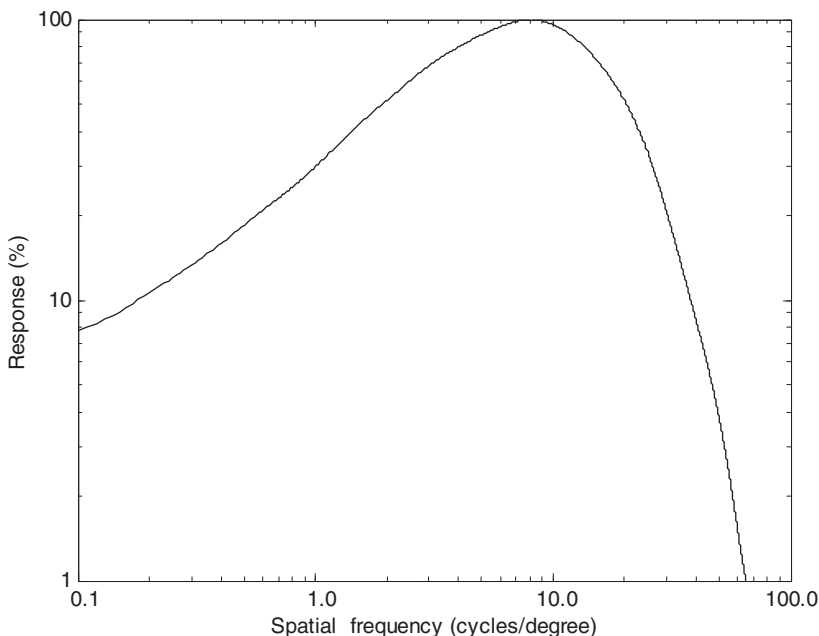


Figure 4.19 Relative spatial frequency response of human visual system.

8	16	19	22	26	27	29	34
16	16	22	24	27	29	34	37
19	22	26	27	29	34	34	38
22	22	26	27	29	34	37	40
22	26	27	29	32	35	40	48
26	27	29	32	35	40	48	58
26	27	29	34	38	46	56	69
27	29	35	38	46	56	69	83

Figure 4.20 Typical values of weighting matrix $W_{i,j}$.

block of data, and $W_{i,j}$ is the weighting value for this particular transform coefficient.

The weighting value $W_{i,j}$ increases as the horizontal and vertical frequencies increase. An example of a matrix of weighting values is shown in Figure 4.20.

Thus if the quantizer step size is 16 and the value of $c(4,4)$ ⁸ for a particular intrablock is 75 then the value of the quantized DCT coefficient, noting that the appropriate quantizer matrix value is 32, would be calculated as shown.

$$c_q(4,4) = \text{round}\left(\frac{8 \times 75}{16 \times 32}\right) = \text{round}(1.17) = 1$$

If the coefficient $c(1,1)$ had the same value of the original DCT coefficient and the quantizer step size was unchanged then the quantized DCT coefficient value would be calculated as shown.

$$c_q(1,1) = \text{round}\left(\frac{8 \times 75}{16 \times 16}\right) = \text{round}(2.34) = 2$$

EXAMPLE 4.6

An 8×8 block of data from a picture is shown in Figure 4.21.

- Calculate the two-dimensional DCT of the data.
- Quantize the data using a quantizer step size of 8.
- Quantize the data using a quantizer step size of 8 using the weighting matrix given in Figure 4.20.

91	42	67	72	83	189	245	241
75	74	171	245	240	227	216	221
50	45	75	65	119	228	245	234
72	93	198	246	239	225	214	222
33	58	75	72	155	242	242	229
75	106	215	248	237	223	216	223

Figure 4.21 Picture data for Example 4.6

⁸The DC DCT coefficient would of course be $c(0,0)$.

- (a) The result after a two-dimensional DCT performed in MATLAB after rounding to the nearest integer is shown in Figure 4.22.

1294	-495	-104	0	-22	34	48	7
-66	-13	84	13	-1	30	5	-8
-15	7	26	1	21	24	-1	-4
-35	-7	53	12	-17	-4	-5	1
-10	8	19	-9	6	15	2	-2
-60	-24	71	25	-26	-7	-1	1
-7	19	35	-25	-8	24	11	0
-189	-117	173	99	-76	-31	0	-2

Figure 4.22 Picture data of Figure 4.21 after two-dimensional DCT.

- (b) Quantizing using a quantizer step size of 8 and rounding to the nearest integer yields the quantized DCT coefficients shown in Figure 4.23.

161	-61	-13	0	-2	4	6	0
-8	-1	10	1	0	3	0	-1
-1	0	3	0	2	3	0	0
-4	0	6	1	-2	0	0	0
-1	1	2	-1	0	1	0	0
-7	-3	8	3	-3	0	0	0
0	2	4	-3	-1	3	1	0
-23	-14	21	12	-9	-3	0	0

Figure 4.23 Quantized DCT coefficients—no weighting matrix.

- (c) Quantizing when the weighting factors used in Figure 4.20 are employed yields the results shown in Figure 4.24.

161	-30	-5	0	0	1	1	0
-4	0	3	0	0	1	0	0
0	0	1	0	0	0	0	0
-1	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0
-2	0	2	0	0	0	0	0
0	0	1	0	0	0	0	0
-7	-4	4	2	-1	0	0	0

Figure 4.24 Quantized DCT coefficients—weighting matrix in Figure 4.20 are employed.

$$\begin{bmatrix} +38 & 0 & +4 & -2 & 0 & 0 & 0 & 0 \\ -5 & +1 & 0 & +1 & 0 & 0 & 0 & 0 \\ +7 & +2 & -2 & +1 & 0 & 0 & 0 & 0 \\ +2 & -1 & +1 & -1 & 0 & 0 & 0 & 0 \\ -2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Figure 4.26 Block of quantized DCT coefficients.

+38, 0, -5, +7, +1, +4, -2, 0, +2, +2, -2, -1, -2, +1, 0, 0, 0, +1, +1, 0, 0, 0, 0, 0, -1, 0, 0,
 0,
 0, 0, 0

Figure 4.27 Coefficients for Figure 4.26 after zig-zag scanning.

(0, +38) (1, -5) (0, +7) (0, +1) (0, +4) (0, -2) (1, +2) (0, +2) (0, -2) (0, -1) (0, -2)
 (0, +1) (3, +1) (0, +1) (5, -1)

Figure 4.28 Coefficients of Figure 4.27 after coding into (run,coefficient) pairs.

Code word_(0, +38), Code word_(1, -5), Code word_(0, +7), Code word_(0, +1), Code word_(0, +4),
 Code word_(0, -2), Code word_(1, +2), Code word_(0, +2), Code word_(0, -2), Code word_(0, -1),
 Code word_(0, -2), Code word_(0, +1), Code word_(3, +1), Code word_(0, +1), Code word_(5, -1),
 Code word_{EOB}.

Figure 4.29 Huffman code words used to represent the quantized DCT coefficients of Figure 4.26.

block (EOB) code word. Because the EOB code word is sent with every transmitted block, it occurs quite commonly and so is able to be represented by a short Huffman code word. For the (run,level) pairs given above, the transmitted code words would be as given in Figure 4.29.

The Huffman coding tables for the set of possible (run,level) pairs have been developed by standards bodies and are based on the statistics of typical sequences of video material.

4.5. MOTION-COMPENSATED DCT ENCODERS AND DECODERS

While we have so far only considered the application of the DCT to original pictures, it can also be used to code the prediction difference after motion-compensated

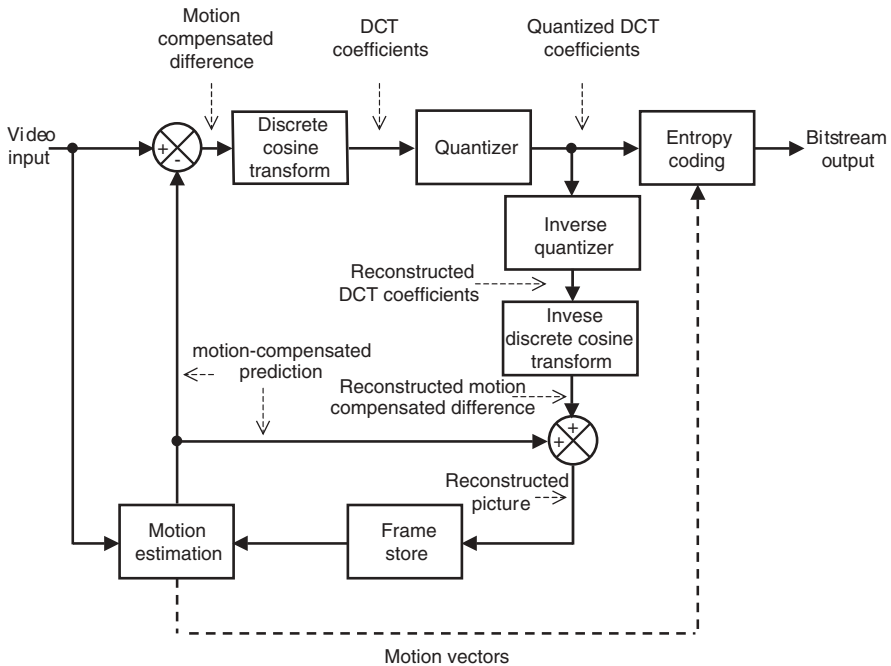


Figure 4.30 Motion-compensated discrete cosine transform encoder.

prediction. Figure 4.30 shows the block diagram of a motion-compensated DCT encoder. The video input has the motion-compensated prediction subtracted from it. The motion-compensated prediction difference is then processed with a two-dimensional DCT prior to quantization. Finally, the quantized DCT coefficients together with the relevant motion vectors are entropy coded and transmitted. The feedback loop of the encoder is equivalent to a decoder and consists of an inverse quantizer⁹ followed by an inverse two-dimensional DCT. This produces a reconstruction of the motion-compensated prediction difference. The motion-compensated prediction is then added to the reconstruction of the motion-compensation prediction difference to form the reconstructed picture, which is stored in a frame store for use in the prediction of a subsequent picture.

The corresponding decoder is shown in Figure 4.31. Apart from the initial entropy decoding stage to produce the transform coefficients and motion vectors, this is identical to the feedback loop of the encoder.

Motion-compensated DCT encoders and decoders are the key coding tools of the MPEG-2 video compression standard that is used for digital television broadcasting. Refinements and improvements introduced during the standardization process significantly enhance the performance of the basic architecture. We will consider this topic in considerable detail in Chapter 6.

⁹However, remember that quantization is a lossy process and so cannot be perfectly reversed.

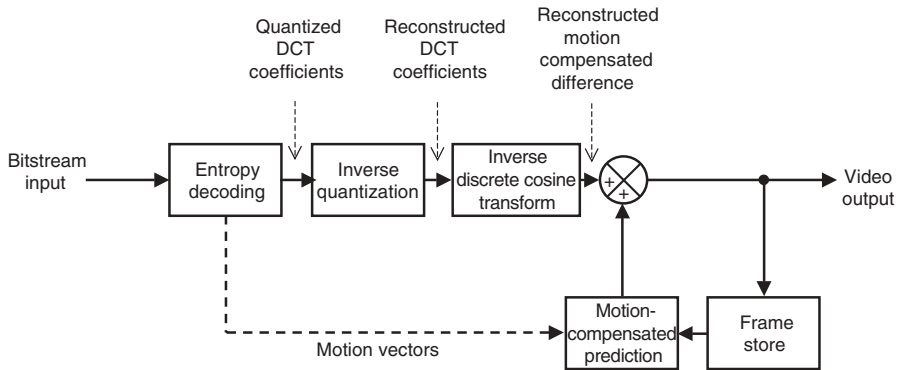


Figure 4.31 Motion-compensated discrete cosine transform decoder.

4.6. RATE CONTROL

The motion-compensated DCT coder shown in Figure 4.32 still has one major problem. Pixels arrive at the encoder at a regular rate and are grouped into 16×16 pixel blocks for motion-compensated prediction. These blocks are subsequently divided into 8×8 pixel blocks for processing with the DCT. Blocks arrive at the motion estimation and the DCT hardware at a regular rate. However, the output of the encoder is inherently a variable rate bit stream. This arises inherently from the fact that

- the number of transform coefficients to be encoded will vary from block to block;
- the number of bits required to encode each transform coefficient is variable and depends on the value of the nonzero coefficients as well as its position within the DCT block (i.e., the number of zeros that precedes each nonzero coefficient).

Most transmission media operate only with constant bit rate (CBR) data streams. It is therefore necessary to turn the variable bit rate (VBR) output of the entropy encoder into a constant bit rate data stream. This process is known as rate control.

The simplest way to achieve rate control is to introduce a buffer between the output of the entropy encoder and the transmission channel. The buffer is simply a block of memory. Data is clocked into the memory at a variable rate and clocked out of the memory at a constant rate in a first in–first out (FIFO) manner. Such a scheme is illustrated in Figure 4.32.

The rate control buffer needs to be sufficiently large to avoid becoming full (called buffer overflow). If the buffer overflows then the data which overflows is lost. This results in serious problems for the decoder in reconstructing the video service.

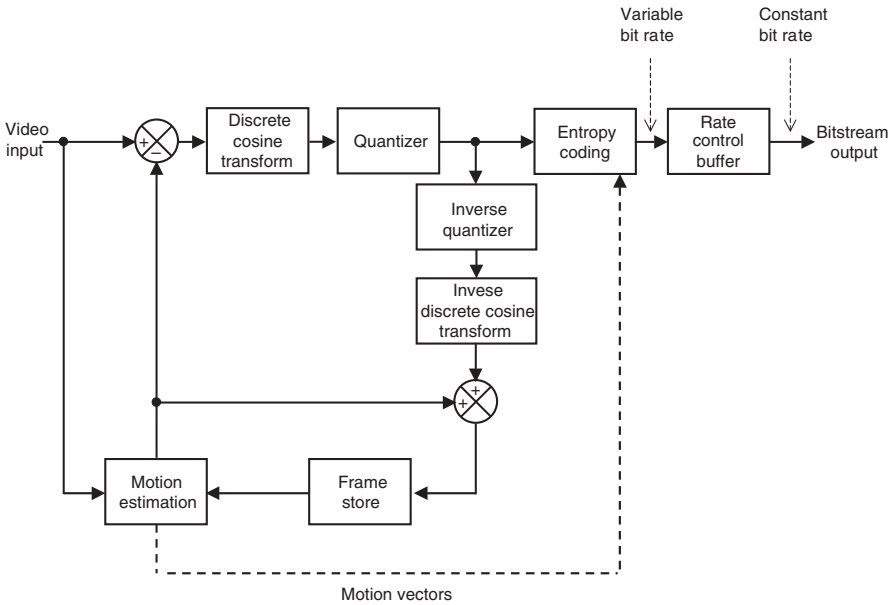


Figure 4.32 Motion-compensated DCT encoder with rate control buffer.

If the rate control buffer was of unlimited size then any variable bit stream could be losslessly transformed to a constant bit rate stream.

In practice, some limit needs to be placed on the size of the rate control buffer. The reason for this becomes clear when it is realized that a second rate control buffer is required at the decoder. In the case of the decoder, the rate control buffer accepts data at a constant rate from the channel but transfers data to the entropy decoder at the variable rate that it requires. Such a decoder is shown in Figure 4.33.

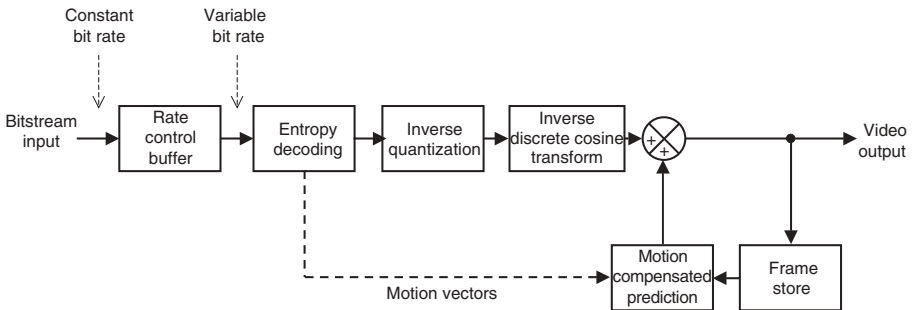


Figure 4.33 Motion-compensated DCT decoder with rate control buffer.

EXAMPLE 4.7

- (a) The rate control buffer of an encoder and a decoder are 12 bits deep each. The average data rate output of the encoder (and hence the average data rate input of the decoder) is 2 bits/s. The channel transmission rate is a constant 2 bits/s. A new code word arrives from the encoder every second. Determine the fullness of each buffer if a series of 3-bit code words arrive consecutively from the encoder. Assume that the encoder and the decoder buffers initially contains exactly two 3-bit code words each. Assume that there is synchronization between the encoder and the decoder so that data is clocked in and clocked out of the buffers at the same rate.
- (b) Repeat this example for the case where a series of 1 bit code words arrive consecutively from the encoder. In this case, assume that the encoder and the decoder buffers each initially contains exactly six 1-bit code words.

For the first case, the state of the two rate control buffers is shown in Figure 4.34.

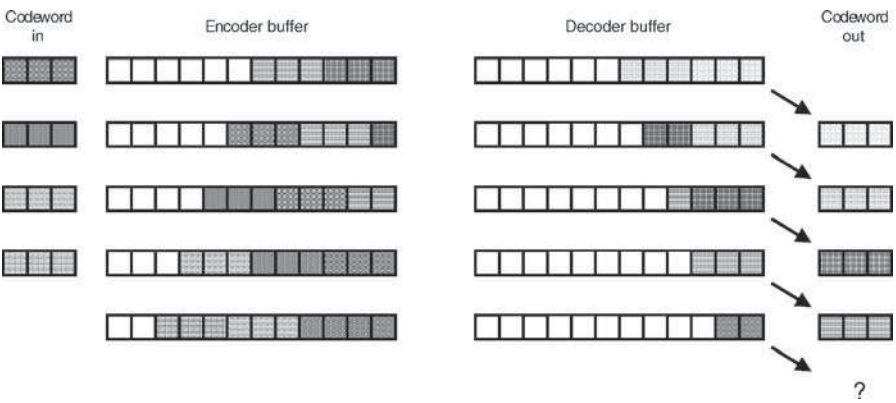


Figure 4.34 Rate control buffer fullness (3-bit code words).

As each code word arrives at the encoder buffer, three new bits are added into the buffer, while two bits are transmitted to the decoder buffer. The amount of data stored in the encoder buffer increases by one bit per code word. At the decoder buffer, two bits are received from the encoder while at the same time a 3-bit code word is read from the buffer. The amount of data stored in the decoder buffer decreases by one bit per code word. Eventually, the amount of data stored in the buffer is insufficient to allow a valid code word to be decoded.

This example is indicative of what happens when the amount of data being generated by the encoder is greater than the transmission rate for a period of time. The encoder buffer fills and the decoder buffer empties. Buffer overflow (at the encoder) and buffer underflow (at the decoder) can occur in this circumstance.

The sum of the bits in the two buffers is indicative of the delay introduced by buffering since

$$\text{Average delay} = \frac{(\text{Encoder buffer fullness} + \text{Decoder buffer fullness})}{\text{Average codeword length}}$$

In this case, the sum of the encoder buffer fullness and decoder buffer fullness is always constant (12 bits).

In the case of 1 bit code words, the situation is as shown in Figure 4.35

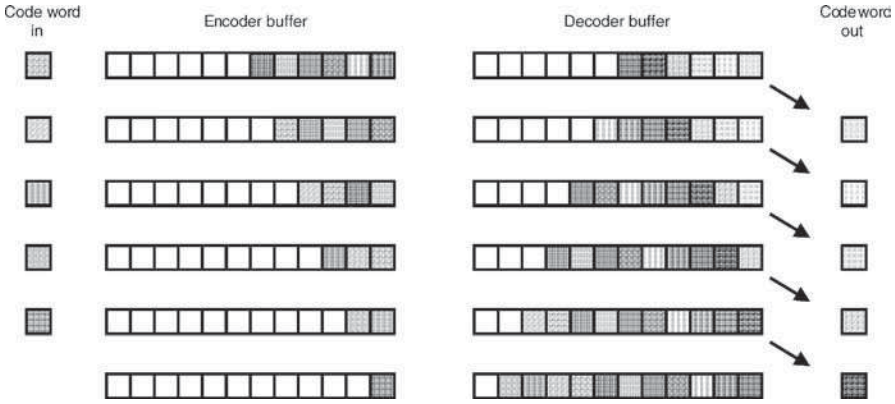


Figure 4.35 Rate control buffer fullness (1-bit code words).

In this case, the decoder buffer fills and the encoder buffer empties. Eventually, a stage is reached where there are insufficient bits in the encoder buffer for the next transmission. Buffer underflow has occurred. At the same time, the decoder buffer is approaching overflow.

Once again, the total number of bits in the two buffers at any time remains constant. ■

In general, the fullness of the encoder buffer at time, $(t + 1)$, $F_e(t + 1)$, can be calculated from the buffer fullness at time, t , $F_e(t)$, according to

$$F_e(t + 1) = F_e(t) + R_{\text{coeff}} - R$$

where R_{coeff} is the number of coefficient bits received from the encoder and R is the number of bits transmitted to the decoder. A similar expression can be developed for the decoder buffer fullness F_d namely

$$F_d(t + 1) = F_d(t) - T_{\text{coeff}} + R$$

where T_{coeff} is the number of coefficient bits passed to the decoder and R is the number of bits received from the encoder. It follows that the total buffer fullness at any time, F_{total} , can be calculated according to

$$F_{\text{total}}(t + 1) = F_{\text{total}}(t) + R_{\text{coeff}} - T_{\text{coeff}}$$

This shows that the total buffer fullness can vary slightly with time. However, because coefficients placed into the encoder buffer are eventually read from the decoder buffer, it follows that over the long term

$$E(R_{\text{coeff}} - T_{\text{coeff}}) = 0$$

The average total buffer fullness is then just the buffer fullness initially. The likelihood of buffer overflow and underflow are minimized if each buffer starts half full. This implies an inherent delay associated with the two buffering processes. The larger the size of the buffers, the larger is this delay. For two-way video services

(i.e., videoconferencing) this delay is crucial, as too much delay leads to substantial difficulties in communicating. For one-way services like digital television, delay is less important. However, the larger the buffer, the more the memory required in the decoder and hence higher the cost. For this reason, some limits are placed on buffer size. This topic is discussed further when we consider the profiles and levels of the MPEG-2 video standard in Chapter 6.

Because buffer overflow and buffer underflow are so undesirable, it is usual to take steps to ensure that they do not occur. One simplistic approach would be to use buffer fullness in determining the quantizer step size to use at the encoder. As the buffer empties, the quantizer step size would be reduced to increase the amount of data that was produced by the encoder. Similarly, as the buffer became full, the quantizer step size would be increased to reduce the amount of data produced by the encoder. In an extreme situation, entire pictures could be dropped completely to ensure that buffer overflow did not occur or dummy data could be transmitted to prevent buffer underflow. Video coding standards support both of these functions, although it is rare that they would need to be used in a high-quality application like digital television. They are commonly used in lower rate, lower quality videoconferencing applications. An encoder incorporating this simple rate control strategy is illustrated in Figure 4.36.

Such an approach, though likely to be successful in avoiding buffer overflow and underflow, is likely to produce a poor quality video service because no account is taken of the characteristics of the human visual system. A good rate controller would consider the likely impact of quantization on particular blocks of the picture from the point of view of a human observer.

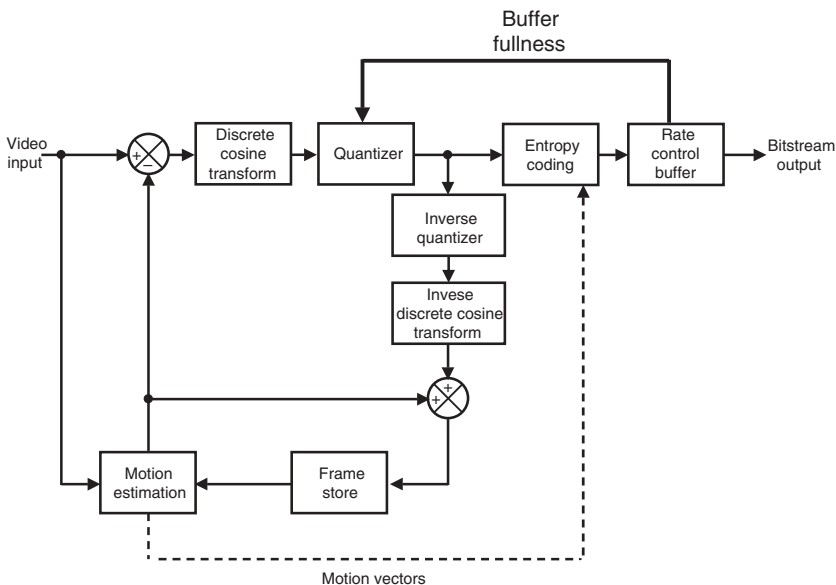


Figure 4.36 A simple example of rate control.

Characteristics of the human visual system that can be taken into account by an effective rate controller include

- *Frequency sensitivity.* As discussed in the section on transform coding, the human visual system is more sensitive to low to medium frequencies than to high frequencies. If an encoder is generating too much data, it makes sense to increase the quantizer step size for higher frequency DCT coefficients first.
- *Visibility threshold.* Coding artifacts are more visible in medium to dark regions of a picture than in brighter regions. Quantizer step size in brighter regions can therefore be increased faster than in darker regions.
- *Spatial masking.* Coding artifacts are far less visible near sharp luminance changes (edges) in pictures than in flat regions. Many rate controllers attempt to measure the “busyness” of a block (i.e., the amount of edge information) in determining the quantizer step size that should be employed.
- *Temporal masking—fast moving objects.* Although the human visual system is good at noticing coding artifacts in stationary objects or objects moving sufficiently slowly that the eye can track them, coding artifacts in fast moving objects are far less perceptible. Blocks containing such objects can be quantized more harshly.
- *Temporal masking—scene changes.* The ability of the human visual system to notice coding artifacts is significantly reduced after a scene change. The first picture of the new scene can therefore be quantized more harshly without affecting subjective quality. This is helpful because the amount of data required to represent the first picture of a new scene can be high due to the ineffectiveness of motion-compensated prediction in this situation.
- *Luminance masking of chrominance.* The human visual system is less able to resolve chrominance information than the luminance information. This is the reason that the two chrominance components are invariably subsampled prior to coding. The human visual system’s ability to resolve chrominance is reduced still further at sharp changes in luminance (i.e., edges).

The effective exploitation of the characteristics of the human visual system is essential if a high-quality digital television service is to be achieved at an acceptable data rate. The topic of how best to do this would form the topic for a book in its own right. Although video compression standards provide tools that can be used to exploit human visual system characteristics, the best means of doing so is not standardized. This allows encoder manufacturers to differentiate their products in the marketplace. Standards bodies only standardize how decoders are to operate. This means that any two compliant decoders presented with the identical compliant bit stream produces an identical decoded video service. However, two compliant encoders when provided with the same video material to encode at the same data rate can produce quite different bit streams and quite different decoded

service qualities depending largely on the effectiveness of their quantization and rate control strategies.

Over the years, much research has been performed in an attempt to define an objective algorithm that is able to measure the subjective quality of a video sequence after coding. The existence of such a measure would greatly ease the task of optimizing the various video coding tools employed by video encoders. Given the complexity of the human visual system, it is perhaps not surprising that this has proved to be an extremely complex task. Peak signal-to-noise ratio (PSNR), as defined in the previous chapter, is known to be inadequate in performing this task. However, its computational simplicity together with its ability to provide reasonable performance comparisons when comparing minor variations of a particular coding algorithm has meant that it is often used for this purpose.

4.7. CONCLUSION

This chapter has introduced the coding tools that form the basis of the MPEG-2 video compression algorithm that is used in current digital television broadcasting standards. Having coded the data, it needs to be transmitted to the decoder in a form that can be understood. This requires the definition of a syntax (basically a set of communication rules—almost a language) for the bit stream. Video syntax the topic of Chapter 5.

PROBLEMS

- 4.1** The pixel vector $[35 \ 17 \ 18 \ 44]$ is to be transformed using the basis functions shown in Figure 4.1. Calculate the resulting transform coefficients. Also perform the inverse transform and hence show that the transform is lossless.
- 4.2** Consider the set of four basis function shown in Figure 4.1.
- (a) Use these basis functions first horizontally and then vertically to obtain the two-dimensional transform of the block of data given below.

60	121	222	247
105	227	240	172
74	186	246	206
184	254	191	212

Comment on the effectiveness of the transform.

- (b) Calculate the pixel values that make up the reconstructed picture when only the top left 2×2 transform coefficients are retained. Also calculate the peak signal-to-noise ratio in decibels.
- 4.3** The horizontal covariance matrix of a picture is shown in Figure 4.37. Calculate the basis functions of the Karhunen–Loeve transform for this picture. Also determine the percentage of energy in each transform coefficient.

$$\begin{bmatrix} 3598 & 3445 & 3170 & 2932 \\ 3445 & 3601 & 3448 & 3172 \\ 3170 & 3448 & 3603 & 3449 \\ 2932 & 3172 & 3449 & 3604 \end{bmatrix}$$

Figure 4.37 Horizontal covariance matrix for Problem 4.3.

- 4.4** The vertical covariance matrix of the same picture used in Problem 4.3 is shown in Figure 4.38. Calculate the basis functions of the Karhunen–Loeve transform for this picture. Also determine the percentage of energy in each transform coefficient.

$$\begin{bmatrix} 3598 & 3121 & 2932 & 2566 \\ 3121 & 3599 & 3122 & 2933 \\ 2932 & 3122 & 3600 & 3123 \\ 2566 & 2933 & 3123 & 3600 \end{bmatrix}$$

Figure 4.38 Vertical covariance matrix for Problem 4.4.

- 4.5** The horizontal covariance matrix of a picture is shown in Figure 4.39.

$$\begin{bmatrix} 1 & a & a^2 & a^3 & a^4 & a^5 & a^6 & a^7 \\ a & 1 & a & a^2 & a^3 & a^4 & a^5 & a^6 \\ a^2 & a & 1 & a & a^2 & a^3 & a^4 & a^5 \\ a^3 & a^2 & a & 1 & a & a^2 & a^3 & a^4 \\ a^4 & a^3 & a^2 & a & 1 & a & a^2 & a^3 \\ a^5 & a^4 & a^3 & a^2 & a & 1 & a & a^2 \\ a^6 & a^5 & a^4 & a^3 & a^2 & a & 1 & a \\ a^7 & a^6 & a^5 & a^4 & a^3 & a^2 & a & 1 \end{bmatrix}$$

Figure 4.39 Horizontal covariance matrix for Problem 4.5.

Where a has the value 0.95. Calculate the basis functions of the Karhunen–Loeve transform for this picture. Compare the result with the basis function of the discrete cosine transform and comment on the result.

- 4.6** Explain why the Karhunen–Loeve transform, despite its optimum performance, is not used in practical image compression systems. Are there any circumstances where the Karhunen–Loeve transform might represent a practical solution?
- 4.7** The Karhunen–Loeve transform is known to concentrate energy into the smallest number of transform coefficients. Does this mean that the coefficients generated will require a minimum number of bits to represent them?
- 4.8** Calculate the basis functions for a 4-point DCT.
- 4.9** Calculate the basis functions for a 16-point DCT.
- 4.10** Calculate the two-dimensional 8-point DCT for the picture data shown in Figure 4.40. Comment on the effectiveness of the DCT in compressing the picture energy into a relatively small number of transform coefficients.

135	145	144	150	152	145	147	149
99	118	135	146	151	142	134	132
142	161	158	142	134	134	153	158
103	113	113	129	147	144	136	138
63	92	124	140	135	131	123	104
59	72	82	103	136	157	163	148
147	150	160	171	172	166	149	128
236	231	211	187	164	149	146	151

Figure 4.40 Picture data for Problem 4.10.

4.11 Consider the picture data given in Figure 4.40

- Perform the two-dimensional DCT and then plot the number of nonzero DCT transform coefficients as a function of quantizer step size.
- Use the weighting matrix given in Figure 4.20 as part of the quantization process and again plot the number of nonzero DCT coefficients as a function of quantizer step size.
- Comment on your results.

4.12 A block of picture data is processed using a two-dimensional DCT followed by quantization. The resulting quantized DCT coefficients are shown in Figure 4.41. Determine the (run,level) pairs that would need to be transmitted to the receiver in order to correctly represent this block.

28	-1	0	0	0	0	0	0
8	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
2	-1	0	0	0	0	0	0
-1	-1	0	0	0	0	0	0
-3	0	0	0	0	0	0	0

Figure 4.41 Quantized DCT data for Problem 4.12.

4.13 If the quantizer step size used to generate the data in Figure 4.41 was 16 and if quantization did not include weighting for the human visual system, calculate the value of the block after reconstruction at the decoder.

4.14 Repeat Problem 4.12 for the quantized DCT data shown in Figure 4.42

212	0	-10	2	10	-1	0	-1
0	0	0	3	-2	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	3	-1	-1	0	0

Figure 4.42 Quantized DCT data for Problem 4.14.

- 4.15 If the quantizer step size used to generate the data in Figure 4.42 was 6 and if quantization did include weighting for the human visual system as defined in Figure 4.20, calculate the value of the block after reconstruction at the decoder.
- 4.16 A decoder receives the (run,level) pair data shown in Figure 4.43 for a particular block of data that has been quantized with a two-dimensional DCT and then quantized using a quantizer step size of 10 with a quantizer that did include weighting for the human visual system as defined in Figure 4.20. Calculate the value of the reconstructed picture data.

(0,119)(0,-5)(0,-1)(0,3)(0,-2)(0,-1)(2,3)(0,1)(1,1)(0,1)(1,1)(33,-1)EOB

Figure 4.43 Received (run,level) pairs for Problem 4.16.

- 4.17 When motion-compensated prediction is combined with the two-dimensional DCT, it is quite common for all quantized DCT coefficients to take the value zero.
- (a) How would such a block be transmitted to the decoder?
- (b) In modern video coders, it is quite common for blocks in which all of the quantized transform coefficients are zero to not to be transmitted at all. They are called skipped blocks. How could such a scheme be made to work given the need for the decoder to be able to unambiguously understand all data transmitted by the encoder?
- 4.18 In what circumstances will a motion-compensated DCT encoder work inefficiently? Suggest methods that might be employed to improve performance. What impact will your improvements have on the way that the encoder and the decoder operate?
- 4.19 A sufficiently large rate control buffer can turn a variable bit rate stream into a constant bit rate stream irrespective of the quantizer used. Explain why such an arrangement is not suitable for real-world applications.
- 4.20 A constant bit rate video encoder is used to transmit a test pattern (i.e., a fixed picture that remains constant for a long period of time). How will the encoder quantizer and rate control buffer react to this situation? What techniques are available to overcome any difficulties?

- 4.21** Most practical video encoder systems spend some time (one or more picture times) analyzing a picture prior to passing it through the motion-compensated DCT process. Explain what is happening during this time. Why is this process important to the provision of high-quality video services?

MATLAB EXERCISE 4.1: EIGENVECTORS OF A PICTURE

In this exercise, we will generate the eigenvectors of a picture and compare the energy compaction ability with that of the discrete cosine transform. Each section should be repeated for a typical picture from the video sequences available to you.

Section 1 Average eigenvectors of a picture

- Calculate the 8×8 horizontal and vertical covariance matrices for the picture. Note that the matrix should be symmetrical about its leading diagonal. These matrices represent the average covariance of the picture and not the exact covariance of each 8×8 pixel block within the picture.
- Calculate the eigenvectors and corresponding eigenvalues that correspond to these covariance matrices. Make sure that you know the appropriate order of the eigenvectors so that maximum energy is compacted into a few coefficients.
- Divide the picture into 8×8 pixel blocks and apply the horizontal eigenvectors to each row within a block, a horizontal Karhunen–Loeve transform. Now apply the vertical eigenvectors to the result of the horizontal KLT. The result is a set of two-dimensional KLT coefficients.
- Plot the percentage of the total picture energy in the top left 1×1 , 2×2 , 3×3 , ..., 8×8 two-dimensional KLT coefficients.
- Repeat the above four steps using the discrete cosine transform instead of the eigenvectors. Make sure that each of the DCT transform vectors is orthonormal. Compare the energy compaction ability of the DCT with that of the eigenvector approach.

Section 2 Individual eigenvectors for every block in a picture

- Repeat Section 1, but this time calculate new covariance matrices (and hence a new set of horizontal and vertical eigenvectors) for each 8×8 pixel block. This represents the optimum transform for the picture. Remember that this would imply the transmission of the eight horizontal and the eight vertical eigenvectors with every block in the picture—a very significant overhead.

In order to get sensible results, the mean of the entire picture (as opposed to the mean of each block) should be subtracted as part of the calculation of the covariance matrix. If the block mean is used, the average value of each block will be exactly zero, which implies that a basis function for the DC value might not be generated (or if it is generated have a relatively low position due to the small amount of

energy it represents and thus the small eigenvalue generated). When a transform is applied to the original picture, in which each block has a nonzero mean, this leads to problems because most of the energy is deleted when the less significant transform coefficients are deleted.

Section 3 Eigenvectors of a difference picture

Repeat Section 1 for an interpicture difference picture produced by subtracting a prediction picture (picture N) from a current picture (picture $N + 1$). Comment on the suitability of the DCT for the coding of prediction difference pictures. If time permits, repeat with a motion-compensated prediction difference picture of the type generated in MATLAB Exercises 3.3 and 3.4.

MATLAB EXERCISE 4.2: DISCRETE COSINE TRANSFORM

Despite the fact that it is suboptimal when compared to the Karhunen–Loeve transform, the discrete cosine transform is used as the basis of transform encoding in modern video encoders. The reasons relate to the fact that the basis functions are fixed and also that performance is close to that of the KLT. In this exercise, we will attempt to quantify the performance of the DCT. In performing this exercise, full use should be made of pictures from the video sequences available to you.

Section 1 The effect of block size on performance

Calculate the two-dimensional DCT of a picture for a range of block size from 2×2 pixels to 64×64 pixels. In each case, set all but the top left quarter of the coefficients to zero (i.e., for an 8×8 pixel block, set all but the top left 4×4 coefficients to zero). Reconstruct the picture and plot the peak signal-to-noise ratio (in decibels) versus block size. Also calculate the number of floating point operations required by MATLAB to perform the transform operation for the various block sizes using the “flops” command. Comment on the effect of block size on the performance of the discrete cosine transform both in terms of operations required and in terms of the reconstructed picture quality.

Section 2 Quantization of DCT coefficients

- Calculate the two-dimensional DCT of a picture using a block size of 8×8 pixels. Quantize the DCT coefficients with a linear quantizer with central dead zone of the type shown in Figure 4.18 for a range of quantizer step sizes and then reconstruct. Calculate the PSNR in decibels for each picture and also examine the quality of the reconstructed picture. Comment on the type of artifacts that are introduced as the quantizer step size increases. Also calculate the entropy of the DCT coefficients (i.e., without the run length coding described in the section on transform encoding) in each case and hence plot a rate distortion curve for the transform encoder.

- Repeat the previous part, but in this case always represent the DC DCT coefficient as a fixed length 8 bit number. This is usually done when a picture is being transform coded without the use of temporal prediction. Comment on the advantages of this approach.

MATLAB EXERCISE 4.3: DISCRETE COSINE TRANSFORM WITH MOTION COMPENSATION

In this MATLAB Exercise we will study the performance of the discrete cosine transform when combined with motion-compensated prediction. In performing this exercise, full use should be made of pictures from the video sequences available to you.

Section 1 The effect of block size on performance

Calculate the motion-compensated prediction of a picture from a picture that occurs temporally earlier in the sequence. Hence calculate the motion-compensated prediction difference.

Calculate the two-dimensional DCT of the motion-compensated prediction difference for a range of block size from 2×2 to 64×64 pixels. In each case, set all but the top left quarter of the coefficients to zero (i.e., for an 8×8 pixel block, set all but the top left 4×4 coefficients to zero). Reconstruct the picture (including reading the motion-compensated prediction) and plot the peak signal-to-noise ratio (in decibels) versus block size. Also calculate the number of floating point operations required by MATLAB to perform the transform operation for the various block sizes using the “flops” command. Comment on the effect of block size on the performance of the discrete cosine transform both in terms of operations required and in terms of the reconstructed picture quality.

Section 2 Quantization of DCT coefficients

- Calculate the two-dimensional DCT of the motion-compensated prediction difference using a block size of 8×8 pixels. Quantize the DCT coefficients with a linear quantizer with central dead zone of the type shown in Figure 4.18 for a range of quantizer step sizes and then reconstruct. Calculate the PSNR in decibels for each picture and also examine the quality of the reconstructed picture. Comment on the type of artifacts that are introduced as the quantizer step size increases. Also calculate the entropy of the DCT coefficients (i.e., without the run length coding described in the section on transform encoding) in each case and hence plot a rate-distortion curve for the transform encoder.